

Lesson 24 — Reuse, Documentation & Refactoring (+ Debug)

Matches LEGO Teacher Portal lesson: "Pattern Party".

Objective (I can...): I can refactor repeated code into functions, document them, and debug a function-based program. **CS concept:** Refactoring; code documentation; debugging modular code · **Builds on:** Lesson 23 (modular design).

AI Connection (2 min): Teams reuse and document shared AI functions so others can build on them safely and give credit. *First-principles question:* "If a bug lives inside a function called in five places, how many places do you fix — and why is that the point?"

Materials: LEGO CS & AI kit + device per pair, a "messy program" card (works but has duplicated code and a planted bug), notebooks. **Model reuse:** Functions model from Lesson 21 — code only.

60-minute flow - Warm-up (8): "Spot the duplicate" — coach shows a program with the same chunk pasted twice; pairs mark the duplication that should become a function. Introduce *refactoring*: improving structure without changing behavior. - **Concept + demo (10):** Take a working-but-messy program, refactor the duplicated chunk into one function, re-run to prove behavior is unchanged, and add a one-line comment/label documenting the function. (*Rotate format:* "coach ships a bug, team catches it.") -

Build/Code (30): Pairs receive the messy program. They (1) refactor duplicated blocks into a function, (2) document each function with a short note of its job, (3) find and fix the single planted bug — fixing it once inside the function. They log the refactor and the fix in the notebook. *Swap at 15 min.* - **Challenge (7):** Parameterize the refactored function so it also replaces a near-duplicate that differed by one value. -

Share + pack-up (5): Pairs show before/after structure and report the bug and where one fix solved it everywhere. Model stays built.

Pair roles: Driver refactors/codes, Navigator documents and leads the debug. *Swap at 15 min. (Trio: third owns documentation.)*

Differentiation: *Support* — point to the duplicated chunk to extract. *Extension / fast-finisher* — refactor two separate duplications. *Reteach* — extract the function together, re-run to prove sameness, then fix the bug.

Exit check: Pair refactors duplicated code into a documented function, behavior unchanged, and fixes the bug in one place. **At-home (optional):** Find repeated instructions in a recipe or manual; rewrite them as one named, documented step. ~10 min.